



Education

Lab 5.3. Migrating an Existing Deployment to Argo Rollouts

Chances are, that you are not starting with a fresh Kubernetes installation but already have a running cluster with deployed workloads. Argo Rollouts has this scenario in mind and provides a migration path to migrate Deployments to Rollout resources.

Objective

Migrate a vanilla Kubernetes Deployment to an Argo Rollout resource.

Prerequisites

- Kubernetes cluster with an argo-rollouts controller.
- kubectl with an argo-rollouts plugin (optional).

Transitioning to Argo Rollouts

1. Preparing resources

For this lab, we will create an NGINX deployment—a task you may have already undertaken numerous times. Run the command below:

```
$ kubectl create deploy nginx-deployment --image=nginx --replicas=3
```

Now check our running pods and deployments using the following command:

```
kubectl get po,deploy
```

Output:

NAME	READY	STATUS	RESTARTS
AGE			
pod/nginx-deployment-7457467ffd-8r8jq	1/1	Running	0
9s			
pod/nginx-deployment-7457467ffd-s79v2	1/1	Running	0
9s			
pod/nginx-deployment-7457467ffd-s9b4h	1/1	Running	0
9s			

NAME	READY	UP-TO-DATE	AVAILABLE	
AGE				
deployment.apps/nginx-deployment	3/3	3	3	9s

2. Convert Deployment to Rollout

Now we want to use the deployment definition to reference it in a new rollout. Run the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  workloadRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {duration: 10s}

EOF
```

Note the field `workloadRef`, which references the `nginx-deployment` resource.

As a result, we have 6 nginx instances running, 3 managed by our vanilla deployment, 3 by the newly created rollout. Run the command below:

```
kubectl get ro,deploy,po
```

Output:

NAME	DESIRED	CURRENT	UP-TO-DATE
rollout.argoproj.io/nginx-rollout	3	3	3

NAME	READY	UP-TO-DATE	AVAILABLE
deployment.apps/nginx-deployment	3/3	3	3

NAME	READY	STATUS	RESTARTS
pod/nginx-deployment-7457467ffd-8r8jq	1/1	Running	0
pod/nginx-deployment-7457467ffd-s79v2	1/1	Running	0
pod/nginx-deployment-7457467ffd-s9b4h	1/1	Running	0
pod/nginx-rollout-6d7df6cfcb-4snrs	1/1	Running	0
pod/nginx-rollout-6d7df6cfcb-bm46n	1/1	Running	0
pod/nginx-rollout-6d7df6cfcb-jg7p8	1/1	Running	0

3. Scale Down Deployment

To finish the migration, we now need to manually scale down the deployment. Run the following command:

```
$ kubectl scale deployment/nginx-deployment --replicas=0
```

Output:

NAME	DESIRED	CURRENT	UP-TO-DATE
rollout.argoproj.io/nginx-rollout	3	3	3

NAME	READY	UP-TO-DATE	AVAILABLE
------	-------	------------	-----------

```
deployment.apps/nginx-deployment 0/0 0 0
12m
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-rollout-6d7df6cfcb-4snrs 9m58s	1/1	Running	0	
pod/nginx-rollout-6d7df6cfcb-bm46n 9m58s	1/1	Running	0	
pod/nginx-rollout-6d7df6cfcb-jg7p8 9m58s	1/1	Running	0	

This leaves you with an up-and-running workload, managed by a rollout resource!

The step of scaling down the deployment once referenced by the Rollout resource can be taken over by the Argo Rollout controller. A special `scaleDown` parameter exists that enables administrators to specify how the deployment should be scaled down (`never`, `onsuccess`, `progressively`).

After confirming the deployment is scaled down, scale it up one more time with the following command:

```
$ kubectl scale deployment/nginx-deployment --replicas=3
```

And apply the following new rollout spec which includes the `scaleDown` parameter:

```
$ cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  workloadRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
    scaleDown: onsuccess
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {duration: 10s}
```

EOF

This will provide the same result as before, except this time there was no need for any manual intervention!

Command:

```
~$ kubectl get ro,deploy,po
```

Output:

NAME	DESIRED	CURRENT	UP-TO-DATE
rollout.argoproj.io/nginx-rollout	3	3	3
12m			

NAME	READY	UP-TO-DATE	AVAILABLE
deployment.apps/nginx-deployment	0/0	0	0
15m			

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-rollout-6d7df6cfcb-4snrs	1/1	Running	0	12m
pod/nginx-rollout-6d7df6cfcb-bm46n	1/1	Running	0	12m
pod/nginx-rollout-6d7df6cfcb-jg7p8	1/1	Running	0	12m

More details on the feature can be found on the [Rollout Migration documentation page](#).

4. Clean Up Resources

Make sure to leave the cluster nice and clean.

Command:

```
$ kubectl delete rollout nginx-rollout
```

Output:

```
rollout.argoproj.io "nginx-rollout" deleted
```

Command:

```
$ kubectl delete deployment nginx-deployment
```

Output:

```
deployment.apps "nginx-deployment" deleted
```